정연서

Software Engineer

yeonseosla@gmail.com github.com/yeonseo-Jung yeonseo-jung.github.io

linkedin.com/in/yeonseo-jung 010-6632-0568

병역 산업기능요원(보충역) 편입 희망

> **비즈니스 문제를 기술로 풀어 실질적 가치로 연결**하는 데 집중하는 소프트웨어 엔지니어입니다. 분산 시스템을 중심으로 **1,600+ 계좌가 동작하는 24/7 운용 환경**에서 시스템 신뢰성·메시징·관측성·장애 대응 구조를 설계·운영해왔으며, **메시징 신뢰성 패턴 설계 · 무중단 언어 마이그레이션 · 레거시 아키텍처 재설계** 같은 시스템 수준 의사결정을 직접 책임진 경험을 보유하고 있습니다.
> 특정 스택에 종속되기보다 **문제에 맞는 도구를 빠르게 검증하고 운영 환경에 적용 가능한 수준까지 내재화**합니다. 설계 결정을 **성능 · 안정성 · 비용 · 복잡도의 트레이드오프**로 해석하며, 도메인이 바뀌어도 재사용 가능한 구조적 사고를 지향합니다.

Problem-First Design

도메인의 핵심 모델과 제약 조건을 먼저 정의한 뒤, 이를 시스템 경계와 데이터 흐름에 반영합니다.

End-to-End Ownership

운영 중 발견한 비효율과 리스크를 개선 과제로 전환하고, 합의된 항목은 설계부터 배포까지 책임지고 추진합니다.

Evidence-Based Decision

관성적으로 사용되던 로직도 데이터와 통계 검정으로 재평가하고, 유효성이 낮은 경우 제품 요구사항과 시스템 구조를 함께 재설계합니다.

SKILLS & STACK

핵심 기술

Python	Expert
Rust	Advanced
Apache Airflow	Advanced
FastAPI / Django	Advanced
RabbitMQ / Celery	Advanced
MySQL / Redis	Advanced
Docker / AWS	Advanced

학력

서울시립대학교

신소재공학과 · 4학년 중퇴

이후 5년간 데이터 엔지니어 및 백엔드 개발자로 실무 + CS 자기 학습 (분산 시스템 · 비동기 런타임 · 메시징 패턴 · DB · 시스템 설계 · Rust 생태계)

분산 시스템 · 메시징

pub/sub · Fanout | DLQ · 멱등성 | Bulkhead · Strangler Fig | Feature Flag · 무중단 배포 | 서킷 브레이커 · 백프레셔

백엔드 · API

FastAPI | Django | Celery | RESTful API 설계

RUST 생태계

tokio (async runtime) | axum (HTTP) | sqlx (async DB) | lapin (AMQP) | Cargo workspace

DEVOPS · 관측성

Docker Compose | Prometheus · Grafana | GitHub Actions

EXPERIENCE

STOA Investment (퀀터스)

2023.01 – 현재 · 3년 5개월

백엔드 · 데이터 파이프라인 · ML 엔지니어링 · 퀀트 시스템

> 초기 멤버로 합류해 백엔드 · 데이터 파이프라인 · 인프라 · 배포 · 운영을 기획부터 고도화까지 전담으로 담당해왔습니다. 주요 프로젝트는 아래와 같고, 각각의 문제 정의와 해결 과정은 Core Projects와 포트폴리오에서 다룹니다.

→ **분산 트레이딩 · 주문 집행 엔진 Zero-to-One 설계 · 운영 — 시그널 생성부와 집행부를 pub/sub 1:N 구조로 물리적 분리**해 부분 장애가 전체로 확산되지 않는 구조를 확보. 메시징 신뢰성 패턴(DLQ 격리 · 시간 슬롯 멱등성 · Bulkhead) 직접 설계.

1,600+ 계좌 | 6개 외부 시스템 통합 | 24/7 무중단 · 메시지 유실 0건 | 월평균 거래대금 5천억

→ **거래 실행 엔진 Python → Rust 무중단 마이그레이션** — 운영 중인 시스템의 **GIL · 이벤트 루프 · 드라이버 호환성을 언어 수준에서 해소**하기 위해 전담 마이그레이션. Strangler Fig + Feature Flag 기반 점진 전환으로 무중단·즉시 롤백 보장. 메모리·이미지·LOC 모두 실측 검증. (상세: Core Project 2)

메모리 -95% (4.3GB → 193MB) | 이미지 -87% (1.2GB → 160MB) | 트레이딩 핵심 LOC -53%

→ **자산배분형 액티브 전략 엔진 전담 설계·구현·운용 — 시장 상태 분류 (regime detection) · 자산별 리스크 가중 배분 (risk parity) · 동적 비중 조절 · 꼬리 위험 자동 차단 (tail-risk hedging) · 추세 신호 반영**의 5개 알고리즘 모듈을 단일 엔진에 결합하고, 백테스팅 시뮬레이터부터 라이브 운용 파이프라인까지 직접 구축.

· 전략 버전 진화(자산 구성 정제 · 신규 자산군 편입)를 무중단 배포로 반영하는 운영 파이프라인까지 직접 책임

· 설계 결정은 walk-forward (시간순 점진 검증) · leave-one-out · 모듈 분리 민감도 · 시뮬레이션↔라이브 일관성의 4단계 검증으로 사후 입증

시뮬레이션 4.15y · 라이브 1.36y · 정합 검증 | 1,600+ 계좌 라이브 운용 무사고 | 매출 40% 기여 · 단일 전략 최대 운용 자산(AUM) 50억

→ **다중 전략 라인업 개발·운용** — 동일한 시뮬레이션·라이브 인프라 위에 입력 신호와 자산 구성이 서로 다른 전략들을 시리즈로 확장.

· **추세 신호 기반 비중 조절 전략군**(지수·섹터·개별주 13종) · **자산 페어 간 동적 비중 스위칭 전략**(2종) 추가 운용

· 각 전략은 가설 정의 → 하이퍼파라미터 walk-forward 최적화 → out-of-sample 일반화 검증 → 라이브 진입의 동일한 파이프라인을 통해 출시

→ **레거시 데이터 파이프라인 Airflow 기반 현대화** — 3년간 직접 유지보수하며 누적된 구조적 부채(오케스트레이션 · 의존성 관리 · 관측성 · 관심사 분리)를 아키텍처 차원에서 진단하고 **Airflow 기반 통합 플랫폼으로 전담 재설계**. 금융 도메인 특수성(거래 세션 · 외부 API 제약)을 스케줄링 계층에 명시 반영, 무중단 전환 후 레거시 2종 완전 폐기.

→ **Django · FastAPI 기반 백엔드 서버 개발 및 운영** — 핀테크 플랫폼의 RESTful API 서버를 전담 설계·구현·운영.

· **동기·비동기 워크로드를 명세별로 분리** — Django DRF로 동기 처리 / FastAPI + asyncio로 비동기 처리를 담당시키고, Celery 기반 백그라운드 잡 큐와 SQLAlchemy 커넥션 풀을 표준화

· 고부하 환경에서의 **안정성 · 관측성 · 롤백 가능성**을 갖춘 운영 체계 확립

Apache Airflow | SQLAlchemy | RabbitMQ · Celery | FastAPI | Rust · tokio · sqlx · lapin | Cargo workspace | Docker Compose

AWS S3 · Azure Blob | ML 서빙 (학습→예측→재학습)

마이셀럽스 (현 키토크에이아이)

2021.12 – 2022.12 · 1년

데이터 엔지니어

> AI 기반 인플루언서 매칭 플랫폼에서 데이터 수집·정제·분석 전 과정의 자동화와 운영 도구를 구축했습니다.

- **데이터 처리 파이프라인 전 과정 자동화** — 라벨링·클렌징·수집·분석을 수동으로 분산 수행하던 업무의 병목을 분석한 뒤, 반복 가능한 단계로 분해해 Python 파이프라인으로 통합. **4~5명 인력 → 1명**으로 효율화 80% 인력 절감
- **비개발자가 운영 가능한 구조로 인수인계** — 개발자가 계속 붙어있어야 돌아가는 파이프라인은 장기적으로 실패한다고 보고, PyQt 기반 UI로 제어·상태·모니터링 기능을 감싸 분석가·운영팀이 직접 파이프라인을 쓰고 관리할 수 있게 만들었습니다.

Python | PyQt | 데이터 파이프라인 | 자동화 | 데이터 품질

CORE PROJECTS

아래는 각 프로젝트의 핵심 요약입니다. **아키텍처 다이어그램 · 대안 비교 트레이드오프 · 설계 결정 근거 · 솔직한 한계** 등 상세 분석은 포트폴리오에서 다룹니다 → yeonseo-jung.github.io/portfolio/swe

1. 핀테크 플랫폼 — 액티브 포트폴리오 운용 시스템 전담 구축

STOA · 2024 – 현재 · 기획·설계·운영

문제

플랫폼 학습 곡선이 높아 라이트·신규 유저 이탈. 기관급 올웨더 프레임워크 (all-weather portfolio)를 개인 투자자가 바로 운용하기 어려워 실전 운용 가능한 형태로 재설계 필요

설계

3-클릭 구독형 B2C 서비스 기획·구현 — 5개 알고리즘 모듈을 단일 엔진에 결합한 **자산배분 운용 시스템**으로 재구성.

· 시장 상태 분류 (regime detection) · 자산별 리스크 가중 배분 (risk parity) · 꼬리 위험 자동 차단 (tail-risk hedging) · 추세 신호 반영

· 전략 버전 진화(자산 구성 정제 · 신규 자산군 편입)를 **무중단 배포**로 반영하는 운영 파이프라인까지 일관 구축

성과

1,600+ 계좌 라이브 운용 1.36년 무사고 · 시뮬레이션↔라이브 risk-adjusted 지표 일관 검증 · 단일 전략 최대 운용 자산(AUM) 50억 · 서비스 전체 매출 40% 기여

· 시뮬레이션 (5.94y / 4.15y) — risk-adjusted 지표에서 벤치마크 대비 일관 우위, drawdown 약 70% 감소

· 라이브 운용 (1.36y) — drawdown 약 44% 감소 + 시뮬레이션↔라이브 out-of-sample 일관성 검증 (정량 수치는 포트폴리오 문서 참조)

Portfolio Optimization | Regime Detection | Risk Overlay | Feature Engineering

2. 분산 트레이딩 시스템 — Zero-to-One 구축 + Rust 마이그레이션

STOA · 2023 – 현재 · 설계·구현·운영

문제

① 계좌 수 증가로 **시그널 생성과 주문 집행이 동일 프로세스에 묶여 장애 파급이 광범위.**

② 24/7 이벤트 매매 코인과 장 세션 · 리밸런싱 기반 주식을 **단일 엔진으로 처리하기 어려움**

설계

시그널 생성부와 주문 집행부를 pub/sub 1:N 구조로 물리적 분리.

· 집행부는 공통 Celery 인프라 위에서 **코인 = 이벤트 기반 / 주식 = 요청 기반 TWAP (Time-Weighted Average Price)**으로 오케스트레이션 이원화

· ML 서빙(학습 → 예측 → 재학습)을 시그널 파이프라인에 통합

성과

1,600+ 계좌 · 6개 거래소/증권사 · 24/7 무중단 · 메시지 유실 0건

· 월평균 거래대금 약 5천억 규모 운용

진화

운영 중 노출된 GIL · 이벤트 루프 · 드라이버 호환성 등 **언어 수준 한계를 해소하기 위해 거래 실행 엔진을 Python → Rust로 무중단 마이그레이션.**

· Strangler Fig + Feature Flag 기반 점진 전환 / 13-crate Cargo workspace 설계 / **거래소 어댑터 trait으로 4개 거래소 단일 계약 통일**

· **메모리 -95% · 이미지 -87% · 트레이딩 핵심 LOC -53%** 실측

RabbitMQ | FastAPI · Celery | asyncio | Rust · tokio · sqlx · lapin | axum | 3-Stage Docker | ML 서빙 (학습→예측→재학습)

3. 데이터 파이프라인 · Airflow 현대화 — 레거시 재설계 및 무중단 전환

STOA · 2026.03 · 진단·재설계·전환

문제

3년간 유지보수하며 축적된 구조적 한계(스케줄러 SPOF · DAG 의존성 부재 · 관심사 미분리 · 관측성 부재)가 장기 장애로 표면화. 개별 수정이 아닌 아키텍처 재설계가 필요한 시점

설계

Airflow 기반 통합 플랫폼으로 이관. 모든 기술 선택은 **"더 좋은 도구"가 아니라 "현재 규모·제약에 맞는 최소 복잡도"** 기준으로 대안 대비 트레이드오프 명시 후 결정

성과

프로덕션 전환 완료, **레거시 2종 완전 폐기.** 장애 복구·백필·회귀 검증 자동화

Apache Airflow | SQLAlchemy | Docker Compose | AWS S3 · Azure Blob